Supplemental Information for Project Plan: GPU Acceleration for Deep Learning

Esther Toobian

May 26, 2026

1 Purpose and Scope

This supplemental document provides additional detail for the proposed project:

GPU Acceleration for Deep Learning:
Tensor Parallelism, Memory Movement, and Hardware Scale

The project studies when GPU acceleration is useful for deep-learning computations, and when the expected benefit may be reduced or eliminated by overhead. The focus is not a broad survey of GPU hardware. Instead, the project is centered on course-connected ideas: tensors, batched linear layers, convolutional layers, computational graphs, autograd, PyTorch modules, and mini-batch training.

The central question is:

> *How do deep-learning tensor computations behave across local CPU, local GPU, and cluster GPU settings?*

The central claim is:

> GPUs help most when the computation is large, parallel, vectorized, batched, and kept on the GPU. They may not help when overhead dominates, such as for small tensors, Python loops, repeated CPU-GPU transfers, insufficient batch sizes, or memory-bound computations.

This scope is intended to keep the project focused. The presentation will not attempt to cover all GPU architecture, CUDA programming, or distributed training. Instead, the emphasis will be on understanding how the mathematical structure of neural-network computations interacts with practical device placement and hardware behavior.

2 Relation to Fleuret's GPU Lecture

Fleuret's Lecture 6.6, "Using GPUs," motivates GPUs by noting that modern deep networks make computation a critical issue, especially for training and hyperparameter optimization. He emphasizes that GPUs were originally developed for real-time graphics, but their highly parallel architecture is well suited to signal processing and high-dimensional linear algebra [1].

Fleuret also presents a practical CPU/GPU memory model. A CPU has access to CPU memory, while a GPU has its own GPU memory. Communication inside GPU memory is fast, while communication between CPU memory and GPU memory is comparatively slow. This directly motivates one of the central themes of the project: GPU acceleration depends not only on arithmetic speed, but also on data movement [1].

The project will follow and extend Fleuret's discussion in three ways:

1. It will connect the GPU discussion back to earlier course material on tensor operations, linear layers, convolutions, computational graphs, and autograd.
2. It will include experiments comparing local CPU, local laptop GPU, and, if available, a PSU research-cluster GPU.
3. It will provide practical guidance on PyTorch device handling, timing methodology, and common situations where GPU use is not beneficial.

3 Mathematical Basis for GPU Acceleration

3.1 Matrix Multiplication

A basic operation in deep learning is matrix multiplication. For $C = AB$,

$$C_{ij} = \sum_k A_{ik} B_{kj}.$$

Each C_{ij} is a dot product, and many output entries can be computed independently or in parallel blocks. This is one of the simplest mathematical reasons GPUs can be effective. Matrix multiplication exposes many similar arithmetic operations. The important point is not that a GPU makes one multiplication intrinsically faster, but that it can perform many similar operations simultaneously when enough parallel work is available.

3.2 Batched Linear Layers

For a batch of N samples,

$$X \in \mathbb{R}^{N \times d_{\text{in}}}, \qquad W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}.$$

A linear layer computes

$$Y = XW^T + \mathbf{1}b^T, \qquad Y \in \mathbb{R}^{N \times d_{\text{out}}}.$$

This connects directly to the course's treatment of batch processing. A single input may not expose much parallelism, but a mini-batch creates many simultaneous dot products. Thus, batch size can strongly affect GPU utilization.

3.3 Convolutional Layers

A convolutional layer computes many local dot products. In simplified notation,

$$Y_{c,i,j} = b_c + \sum_{d,u,v} K_{c,d,u,v} X_{d,i+u,j+v}.$$

For each output channel c and spatial location (i, j), the layer computes a local dot product between a kernel and a receptive field. Since this is repeated across many channels and spatial positions, convolution is computationally expensive but highly parallelizable.

3.4 Backpropagation

The backward pass also consists of tensor operations. Gradients for linear and convolutional layers can be expressed using matrix products, tensor contractions, and convolution-related operations. Thus, GPU acceleration is relevant to training, not only inference.

4 CPU/GPU Memory and Device Model

A CPU and a GPU are optimized for different types of computation.

CPU	GPU
Fewer, more flexible cores	Many simpler parallel cores
Sequential/control-flow tasks	Many similar arithmetic operations
Low-latency execution	High-throughput execution
Direct access to CPU RAM	Separate GPU memory / VRAM
Often effective for small tasks	Most effective for large tensor operations

The key practical issue is memory movement. If data repeatedly moves between CPU memory and GPU memory, transfer cost may dominate computation. This is why GPU acceleration is not automatic.

5 PyTorch Device Semantics

In PyTorch, tensors live on devices. Operations are performed on the device where the tensors reside. A typical device-aware pattern is:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = model.to(device)
criterion = criterion.to(device)

train_input, train_target = train_input.to(device), train_target.to(device)
test_input, test_target = test_input.to(device), test_target.to(device)
```

Important rules include:

- CPU tensors are computed on the CPU.
- GPU tensors are computed on the GPU.
- Operands in the same operation generally must be on the same device.
- `tensor.to(device)` returns a tensor on the requested device. If the tensor is not already there, this requires a copy.
- `module.to(device)` moves parameters and buffers recursively.
- Moving tensors between CPU and GPU memory should be done carefully and not inside tight loops unless necessary.

A useful demonstration from Fleuret's discussion is that moving a CPU tensor to the GPU creates a copy, while moving a CPU tensor to the CPU may return the same object or storage. This can be used to show that device movement is not just a label change. It affects where data is stored and how operations are executed [1].

6 Software Stack: PyTorch, CUDA, and Backend Libraries

The project will briefly discuss the software stack that connects high-level PyTorch code to low-level computation. In particular, Fleuret notes that deep-learning frameworks interface with computational backends such as BLAS, LAPACK, cuBLAS, and cuDNN [1].

- **PyTorch:** The user-facing deep-learning framework used to define tensor operations and neural-network modules
- **CUDA:** NVIDIA's platform and programming model for GPU computation [4]
- **BLAS:** Basic linear algebra routines, especially vector and matrix operations
- **LAPACK:** Higher-level numerical linear algebra routines, including solves and eigendecompositions
- **cuBLAS:** NVIDIA's GPU-accelerated BLAS-style library [5]
- **cuDNN:** NVIDIA's library for GPU-accelerated deep-learning operations such as convolution, pooling, and normalization [6]

The main point is that PyTorch is not executing large tensor operations through slow Python loops. When operations are expressed in vectorized tensor form, PyTorch can dispatch them to optimized compiled kernels. This is one reason vectorized tensor code can benefit from GPU acceleration while Python loops often do not [1, 5, 6].

7 Experimental Platform

Local experiments will be run on a Lenovo Legion Pro 7 16IAX10H Type 83F5 equipped with an NVIDIA GeForce RTX 5090 Laptop GPU. The current local Python/PyTorch environment reports the following configuration:

Item	Local value
Machine	Lenovo Legion Pro 7 16IAX10H Type 83F5
Python	3.11.14
Platform	Windows 10 / build 10.0.26200
PyTorch	2.11.0+cu128
CUDA available to PyTorch	Yes
CUDA used by PyTorch	12.8
GPU	NVIDIA GeForce RTX 5090 Laptop GPU
GPU memory reported by PyTorch	approx. 23.89 GB
Streaming multiprocessors	82
NVIDIA-SMI driver	592.01
NVIDIA-SMI CUDA support	13.1

The CUDA version reported by PyTorch and the CUDA version reported by `nvidia-smi` differ in this environment. PyTorch reports the CUDA runtime version used by the installed PyTorch build, while `nvidia-smi` reports the CUDA version supported by the installed NVIDIA driver. This distinction is included to make the software environment reproducible [3, 4].

If access and scheduling permit, selected benchmarks will also be run on a PSU research-cluster GPU. For any cluster results, the GPU model, CPU model, PyTorch version, CUDA environment, scheduler configuration, and job settings will be recorded before interpreting the results.

All timing results will be treated as hardware-specific case studies rather than universal benchmark claims. This is especially important because laptop GPU performance can depend on power mode, cooling, background processes, and thermal limits.

8 Planned Experiments

The main experiments will compare:

$$\text{local CPU} \quad vs \quad \text{local laptop GPU} \quad vs \quad \text{cluster GPU}$$

8.1 Experiment 1: Matrix Multiplication Size Sweep

The first experiment compares runtime for

$$A, B \in \mathbb{R}^{n \times n}, \qquad C = AB,$$

as n increases.

Possible sizes include: $n \in \{16, 32, 64, 128, 256, 512, 1024, 2048, 4096\}$

Expected behavior:

- CPU may be competitive or faster for small n.
- GPU should become faster for large n.
- The local laptop GPU and cluster GPU may have different crossover points and speedups.

This experiment is the cleanest demonstration of the tradeoff between overhead and parallel throughput.

8.2 Experiment 2: Batch Size and Neural-Network Throughput

The second experiment runs forward and backward passes through a small MLP or CNN for different batch sizes.

Possible batch sizes include: $1,\ 8,\ 32,\ 128,\ 512,\ 1024,\ 2048$

Expected behavior:

- Batch size 1 may underuse the GPU.
- Larger batches should improve throughput up to a saturation point.
- Very large batches may stop improving or exceed available GPU memory.

This experiment connects directly to mini-batch training and the course's discussion of batch processing.

8.3 Experiment 3: CPU-GPU Transfer Overhead

This experiment compares strategies such as:

1. Move data to the GPU once and perform many operations.
2. Move data from CPU to GPU inside the loop.
3. Move results from GPU to CPU inside the loop.

Expected behavior:

- Moving data once and reusing it should be much faster.
- Repeated transfers can dominate runtime.

This experiment directly demonstrates the memory-movement issue emphasized by Fleuret [1].

8.4 Experiment 4: Vectorized Tensor Operations vs Python Loops

This experiment compares vectorized tensor operations with explicit Python loops.

Example vectorized operation: $C = A + B$

Expected behavior:

- Vectorized tensor operations can use optimized backend kernels.
- Python loops prevent efficient use of GPU parallelism.

8.5 Optional Experiment 5: Multi-GPU DataParallel

If time and cluster resources permit, a small `nn.DataParallel` example will be run. Following Fleuret's description, `nn.DataParallel` splits a mini-batch along the first dimension, sends pieces to model replicas on multiple GPUs, and concatenates the results. It is compatible with autograd [1].

If this experiment is not feasible, multi-GPU behavior will be explained conceptually as an extension.

8.6 Summary of Experiment Goals

Experiment	Main question tested
Matrix multiplication	At what problem size does GPU parallel throughput outweigh overhead?
Batch size	How much parallel work is needed to use the GPU effectively?
Transfer overhead	When does CPU-GPU memory movement dominate computation?
Vectorization vs loops	Why do tensor kernels benefit from GPUs while Python loops often do not?
DataParallel	How does splitting a mini-batch across multiple GPUs change the computation?

9 Timing Methodology

GPU timing must be handled carefully since CUDA operations are asynchronous. Timing code will include warmup iterations and synchronization before stopping the timer. The call to `torch.cuda.synchronize()` ensures that queued GPU operations have completed before the elapsed time is recorded.

```
import time
import torch

def time_gpu_operation(fn, warmup=10, repeats=50):
    for _ in range(warmup):
        fn()
    torch.cuda.synchronize()

    start = time.perf_counter()
    for _ in range(repeats):
        fn()
    torch.cuda.synchronize()
    end = time.perf_counter()

    return (end - start) / repeats
```

For fair comparison, experiments should include:

- Warmup iterations
- Multiple repeats
- Mean and standard deviation
- Consistent tensor dtype
- Controlled device placement
- No unnecessary printing inside timed loops
- Separation of transfer time and compute time where appropriate

10 Cluster Extension

If time permits, selected experiments will be run on a PSU research-cluster GPU. This adds two useful dimensions to the project:

1. Comparison of local laptop GPU performance to research-computing GPU performance
2. Introduction of practical GPU access in a cluster environment

To access a GPU on the cluster, a job will likely require a scheduler script that requests GPU resources, loads the appropriate software environment, and runs the benchmark script. A simple example, written in SLURM-style syntax, could be introduced as:

```
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=00:10:00

python gpu_benchmark.py
```

The exact scheduler directives will depend on the cluster environment. The project will avoid becoming a full cluster-computing tutorial. The purpose of this extension is to show that access to a GPU through a scheduler is separate from writing PyTorch code that correctly uses the GPU.

11 Scope and Expected Takeaways

The project is structured to connect practical GPU use with the mathematical and computational ideas developed in the course. The main emphasis is on reasoning from the structure of the computation to determine whether an operation is large enough, parallel enough, and organized in a way that can benefit from GPU execution.

The presentation will connect the following ideas:

- The algebraic structure of matrix multiplication and convolution
- Parallelism in tensor operations
- Memory hierarchy and CPU-GPU data movement
- Batching and throughput
- Forward and backward computational graphs
- PyTorch device semantics
- Backend libraries
- Experimental benchmarking methodology
- Local hardware versus cluster hardware

The expected takeaways are:

1. GPUs help most when computation is large, parallel, vectorized, and batched.
2. Matrix multiplication and convolution are GPU-friendly because they expose many repeated dot products.
3. Batching increases parallel work and can improve GPU utilization.
4. Data movement between CPU and GPU can dominate runtime.
5. Small operations may be faster on CPU.
6. PyTorch code must keep tensors, model parameters, and targets on compatible devices.
7. GPU timing requires synchronization.
8. Cluster GPUs may offer different performance characteristics from local laptop GPUs, but they also introduce scheduling and environment considerations.
9. Multi-GPU computation can help, but communication and synchronization overhead mean that scaling is not automatic.

References

[1] François Fleuret. *Deep Learning: 6.6 Using GPUs*. Course handout.

[2] François Fleuret. *Deep Learning: 6.1 Benefits of Depth*. Course handout.

[3] PyTorch. *Get Started Locally*. https://pytorch.org/get-started/locally/

[4] NVIDIA. *CUDA Platform for Accelerated Computing*. https://developer.nvidia.com/cuda

[5] NVIDIA. *cuBLAS*. https://developer.nvidia.com/cublas

[6] NVIDIA. *cuDNN*. https://developer.nvidia.com/cudnn

[7] NVIDIA. *GeForce RTX 50 Series Laptop GPUs*. NVIDIA product documentation.

[8] Lenovo. *Legion Pro 7 16IAX10H Platform Specifications*. Lenovo PSREF.

[9] R. Raina, A. Madhavan, and A. Y. Ng. *Large-scale Deep Unsupervised Learning Using Graphics Processors*. International Conference on Machine Learning, 2009.

[10] A. Krizhevsky, I. Sutskever, and G. Hinton. *ImageNet Classification with Deep Convolutional Neural Networks*. Neural Information Processing Systems, 2012.

[11] S. Shi, Q. Wang, P. Xu, and X. Chu. *Benchmarking State-of-the-Art Deep Learning Software Tools*. 2016.